

AnchorOverlap: Technical Appendix (Supplementary Material)

Anonymous Author(s)

This technical appendix accompanies the submission “AnchorOverlap: Efficient Collective–GEMM Optimization Based on Consumer Dependencies”. Algorithm 2 and Algorithm 3 continue the numbering of the main paper, whose Algorithm 1 appears there. Section references without a letter prefix point to that paper.

A RS/AR Runtime Protocol Pseudocode

Algorithm 2 summarizes the per-rank handoff protocol. A **submit** block queues a task without waiting for completion; producers retain submission order, while transfers and reductions run on their configured streams. $\mathcal{U}$ orders blocks $\omega = (c, b)$ by row chunk and then column band. The slot is $x_\omega = (c \bmod L, b)$ for RS and $x_\omega = (cB + b) \bmod R_{AR}$ for AR; current and predecessor tickets τ_ω, π_ω follow Section 4.2 of the main paper. Ready and done abstract current-block production and local consumption, respectively; their tests become true when the corresponding work completes.

For multi-band RS, $\text{ProduceRS}([0, F])$ computes the first F of the C row chunks for every destination; the next call computes the remainder. Each panel publishes readiness after its tiles are stored; tiles within a batch remain independently schedulable. The single-band path combines these calls into one batch. $\text{Slot}_s[u, x]$ denotes source u ’s receiving region in slot x on rank s . Send copies into that region at the destination and publishes $\text{arrival}_d[s, x] = \tau$ once the data is visible.

AR submits each block’s consumer before its next producer. Exchange sends the current accumulated value to peer v , publishes arrival, and waits for both outgoing-copy completion and the peer’s matching arrival before returning the received value. The subsequent addition therefore cannot overwrite an unfinished send. The XOR loop runs sequentially within a block; its completed consumer releases the slot. Join makes downstream work depend on all submitted tasks.

Algorithm 2: RS/AR handoff with reusable staging

Input: $o \in \{\text{RS}, \text{AR}\}; A_s, W_s, \mathcal{U}, \theta$
Output: D_s

```

1 if  $o = \text{RS}$  then
2   submit producer do
3      $\text{ProduceRS}([0, F]); \text{ProduceRS}([F, C]);$ 
4   foreach  $\omega \in \mathcal{U}$  do
5      $(x, \tau, \pi) \leftarrow (x_\omega, \tau_\omega, \pi_\omega);$ 
6     foreach  $d \neq s$  do
7       submit transfer to  $d$  do
8         await  $\text{Ready}_s[d, \omega] \wedge \text{free}_d[x] = \pi;$ 
9          $\text{Send}(P_{s \rightarrow d}[\omega], d, x, \tau);$ 
10    submit reduction of  $\omega$  do
11      await  $\text{Ready}_s[s, \omega] \wedge \bigwedge_{u \neq s} \text{arrival}_s[u, x] = \tau;$ 
12       $D_s[\omega] \leftarrow P_{s \rightarrow s}[\omega] + \sum_{u \neq s} \text{Slot}_s[u, x];$ 
13       $\text{free}_s[x] \leftarrow \tau;$ 
14 else
15   for  $k = 0, \dots, |\mathcal{U}| - 1$  do
16      $\omega \leftarrow \mathcal{U}[k]; (x, \tau, \pi) \leftarrow (x_\omega, \tau_\omega, \pi_\omega);$ 
17     submit producer do
18       if  $k \geq W_{\text{look}}$  then
19         await  $\text{done}[k - W_{\text{look}}];$ 
20          $P_s[\omega] \leftarrow \text{GEMM}(A_s[C_c, :], W_s[:, B_b]);$ 
21          $\text{Ready}_s[\omega] \leftarrow \text{true};$ 
22     submit reduction of  $\omega$  do
23       await  $\text{Ready}_s[\omega];$ 
24        $D_s[\omega] \leftarrow P_s[\omega];$ 
25       for  $j = 0, \dots, \log_2 p - 1$  do
26          $v \leftarrow s \oplus 2^j;$ 
27         await  $\text{free}_v[x] = \pi;$ 
28          $D_s[\omega] \leftarrow D_s[\omega] + \text{Exchange}(D_s[\omega], v, x, \tau);$ 
29        $\text{free}_s[x] \leftarrow \tau; \text{done}[k] \leftarrow \text{true};$ 
30 Join();
31 return  $D_s;$ 

```

B Pseudocode of the Constraint-Aware Two-Stage Search

Algorithm 3 summarizes the two-stage search using the RS candidate organization in Section 4.3 of the main paper. Representatives fixes initial steady-state/tail resources,

deduplicates structural candidates, groups them by (h_c, L, B) , and uses the structural heuristic to retain at most q representatives per bucket within budget b_1 . `ExpandResources` expands the best measured structures into resource pairs within budget b_2 .

`Measure` allocates the workspace and measures each candidate, returning successful (θ, T) records with $T = \max_s t_s$. Failed allocation or execution produces no record. `Topk` selects at most k fastest configurations. Long measurement increases warmup and timing counts; if no long run succeeds, the algorithm returns the second-stage winner with *validated* = false.

Algorithm 3: Two-stage configuration search

Input: x, H, Θ_{RS} ; budgets b_1, b_2 ; quota q ; shortlist sizes m, v

Output: $(\theta^*, \text{validated})$ or FAIL

```

1  $\Theta \leftarrow \text{NormalizeDeduplicate}(\Theta_{RS}, x, H)$ ;
2  $C_1 \leftarrow \text{Representatives}(\Theta, q, b_1)$ ;
3  $\mathcal{R}_1 \leftarrow \text{Measure}(C_1, \text{short})$ ;
4 if  $\mathcal{R}_1 = \emptyset$  then
5   return FAIL;
6  $\mathcal{K} \leftarrow \text{Top}_m(\mathcal{R}_1)$ ;
7  $C_2 \leftarrow \text{ExpandResources}(\mathcal{K}, b_2)$ ;
8  $\mathcal{R}_2 \leftarrow \text{Measure}(C_2, \text{short})$ ;
9 if  $\mathcal{R}_2 = \emptyset$  then
10  return FAIL;
11  $\mathcal{V} \leftarrow \text{Measure}(\text{Top}_v(\mathcal{R}_2), \text{long})$ ;
12 if  $\mathcal{V} \neq \emptyset$  then
13    $(\theta^*, T^*) \leftarrow \arg \min_{(\theta, T) \in \mathcal{V}} T$ ;
14   return  $(\theta^*, \text{true})$ ;
15 else
16    $(\theta^*, T^*) \leftarrow \arg \min_{(\theta, T) \in \mathcal{R}_2} T$ ;
17   return  $(\theta^*, \text{false})$ ;
```
